PRUNING WITH OCCAM'S RAZOR

Anonymous authors
Paper under double-blind review

ABSTRACT

Deep learning neural network models must be large enough to adapt to their problem domain, while small enough to avoid overfitting training data during gradient descent. To balance these competing demands, over-provisioned deep learning models such as transformers are trained for a single epoch on large data sets, and hence inefficient with both computing resources and training data. In response to these inefficiencies, we derive a provably good algorithm that can combine any training and pruning methods to simultaneously optimize efficiency and accuracy, identifying conditions that resist overfitting and reduce model size while outperforming the underlying training algorithm. We then use the algorithm to combine gradient descent with magnitude pruning into "Occam Gradient Descent." With respect to loss, compute and model size (a) on image classification benchmarks, linear and convolutional neural networks trained with Occam Gradient Descent outperform traditional gradient descent with or without post-train pruning; (b) on a range of tabular data classification tasks, neural networks trained with Occam Gradient Descent outperform traditional gradient descent, as well as Random Forests; (c) on natural language transformers, Occam Gradient Descent outperforms traditional gradient descent.

1 INTRODUCTION

Deep learning models are artificial neural networks often with hundreds of billions of parameters, e.g.,Brown et al. (2020), Rae et al. (2021), Smith et al. (2022), Thoppilan et al. (2022). However, trained models are sparse in that most of the parameters are negligible, raising the question as to whether the models really need to be large to perform, e.g., Kaplan et al. (2020) and Hoffmann et al. (2022) in the context of Large Language Models (LLMs). Kausik (2024) suggests that LLMs are vastly overprovisioned compared to the theoretical estimated dimensionality of the training data. While overprovisioned models can adapt well to the problem domain, they are prone to overfitting and poor generalization, Chang et al. (2021). As a result, large models are typically trained for just a single epoch, Xue et al. (2023).

Our results are related to several categories of work in the literature. Firstly, network pruning, e.g., LeCun et al. (1989) Han et al. (2015), Hassibi et al. (1993), Han et al. (2015), Liu et al. (2018), Blalock et al. (2020),Sun et al. (2023), Hoefler et al. (2021) and Frantar & Alistarh (2023), which set to zero some of the parameters of a trained network in order to reduce model size with minimal loss of accuracy. Secondly, knowledge distillation, e.g., Hinton et al. (2015), Chen et al. (2017) and Asami et al. (2017), which seek a smaller network that mimics a larger network with minimal loss of accuracy, and the application of distillation to regularization, e.g. Yuan et al. (2020), Ghosh & Motani (2021). Thirdly, learning theory, Valiant (1984), Natarajan (1989), Haussler (1992), Shalev-Shwartz & Ben-David (2014), Blumer et al. (1987), Board & Pitt (1990), and Natarajan (1993). Fourthly, regularization, e.g., Tibshirani (1996), Kukačka et al. (2017), which invokes Occam's Razor to minimize the norm of the weights of a neural network by including the norm as an additive term in the loss function for gradient descent. While regularization has empirical benefits, it does not reduce the model size, and including the norm in the loss function creates an ad hoc tradeoff between the training loss and the norm.

Building upon prior work, our Occam Pruning algorithm (named after Occam's Razor: "The simplest explanation is most likely correct") combines any training and pruning methods to simultaneously optimize efficiency and accuracy, identifying conditions for (a) monotonic convergence that resists overfitting (b) model size decreasing exponentially in the loss (c) outperforming the underlying

training algorithm. We then use the algorithm to combine gradient descent with magnitude pruning into "Occam Gradient Descent," which works on any neural network without modifications or limitations such as random graphs or Gumbel softmax, e.g.,Mocanu et al. (2018), Zhang et al. (2023). Furthermore, our algorithm adaptively prunes a network to outperform the original network, improving on the "lottery ticket hypothesis" of Frankle & Carbin (2018), who hypothesize an ad hoc pruning method for comparable performance. We also note that in contrast to the large body of work on Neural Architecture Search, e.g. as surveyed in White et al. (2023), our results are focused on training a given neural network.

With respect to loss, compute and model size, our experiments show (a) on the MNIST and CIFAR10 image classification benchmarks, linear and convolutional neural networks trained with Occam Gradient Descent outperform traditional gradient descent with or without post-train pruning; (b) on a range of tabular data classification tasks, neural networks trained with Occam Gradient Descent outperform traditional gradient descent, as well as Random Forests, e.g.,Sutton (2005), Biau & Scornet (2016); (c) on natural language transformers, Occam Gradient Descent outperforms traditional gradient descent. While the experiments in this paper use magnitude pruning with the Adam and AdamW optimizers for traditional gradient descent, we expect similar performance on other variants per our theoretical results. The scale of our experiments is limited by budget constraints.

2 Theoretical Results

Consider functions of the form $f : X \to [k]$, where X is the domain and $[k] = \{1, 2, ...k\}$ is the range of k labels. A neural network computes a space of functions $F : W \times [k]$, where W is the space of the weights of the network. For a specific choice of weights $w \in W$, the function $f : X \to [k]$ computed by the network is denoted as $f(x) = F(w, x)$, for $x \in X$.

For function $f : X \to [k]$ and a probability distribution P on $X \times [k]$, the discrete loss of f with respect to P is the probability that f is incorrect, i.e.,

$$L(f, P) = P\{(x, y) : f(x) \neq y\} \tag{1}$$

Neural Network Training Problem: Given a collection of training samples $S = \{(x_i, y_i)\}$ drawn on a distribution P, compute $w \in W$ such that $f(x) = F(w, x)$ minimizes $L(f, P)$.

Gradient descent is the established method to solve the above problem. Starting with random initial values for the weights, the training loss $L(f, S)$ is reduced along its steepest gradient on the weights, iterating over the training samples. Each pass across the set of training samples is called an epoch. However, if the training algorithm is run for multiple epochs on the training set, the weights are optimized for a distribution that favors the training samples rather than the natural distribution. This is commonly referred to as overfitting, and leads to poor generalization and low accuracy on test data. To avoid overfitting, neural network models are over-provisioned but trained for just one epoch on the training data, resulting in inefficient use of both computing resources and training data. Towards a theoretical analysis of the above, we examine the relevant learning theoretic results as surveyed in Shalev-Shwartz & Ben-David (2014).

Definition 1. *Given ϵ and δ in (0,1), and samples drawn on probability distribution P, an agnostic learning algorithm finds $f \in F$ with confidence at least $(1 - \delta)$ such that $L(f, P) = \min_{h \in F} L(h, P) + \epsilon$*

There are several measures for the sample complexity of a space of functions that can be used to bound , such as the Vapnik Chervonenkis dimension, the Generalized dimension and Rademacher complexity. Since we need bounds on multi-class functions, we use the notion of the Generalized dimension, Natarajan (1989), also known as the Natarajan dimension, Shalev-Shwartz & Ben-David (2014). For brevity, we will simply refer to it as the dimension.

Definition 2. *(Generalized shattering) A set $C \subset X$ is shattered by F if there exist two functions f_0, f_1 in F such that for every $x \in C$, $f_0(x) \neq f_1(x)$; and for every $B \subset C$, there exists $f_2 \in F$, such that for all $x \in B$, $f_2(x) = f_0(x)$, and for all $x \in (C - B)$, $f_2(x) = f_1(x)$.*

Definition 3. *The dimension of a space of functions F is the size of the largest set shattered by it, and is denoted by $dim(F)$.*

```
_________________________________________________________________
 Layer (type)                Output Shape              Param #
=================================================================
 flatten (Flatten)           (None, 784)               0

 dense (Dense)               (None, 1024)              803840

 dropout (Dropout)           (None, 1024)              0

 dense_1 (Dense)             (None, 10)                10250

=================================================================
Total params: 814090 (3.11 MB)
Trainable params: 814090 (3.11 MB)
Non-trainable params: 0 (0.00 Byte)
_________________________________________________________________
```

Figure 1: Oversized off-the shelf linear network for MNIST

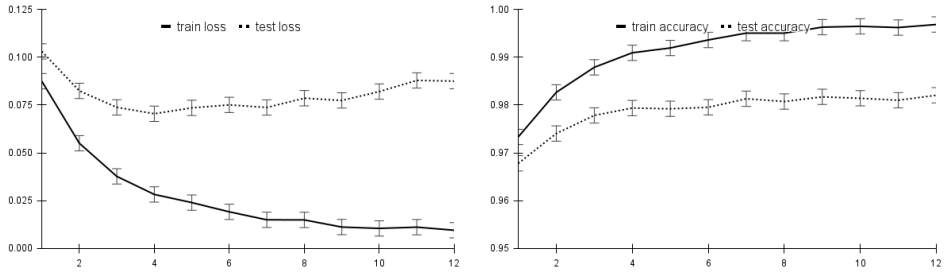


Figure 2: Gradient descent loss & accuracy; MNIST; average of ten runs.

Intuitively, the dimension of a space of functions is a measure of the richness of the space, i.e. the number of degrees of freedom across the functions in the space. The following theorem is adapted from Shalev-Shwartz & Ben-David (2014), and proved therein, based on Haussler (1992).

Theorem 1. *For fixed k and δ, the error in any agnostic learning algorithm for a space of functions F from m samples is* $\Theta[(dim(F)/m)^{0.5}]$

Therefore for fixed k, δ, the expected test loss $L(f, P)$ of an agnostic learning algorithm is of the form

$$L(f, P) = \min_{h \in F} L(h, P) + \Theta[(dim(F)/m)^{0.5}] \tag{2}$$

Equation 2 applies to a neural network. $L(f, P)$ is the expected test loss, $min_{h \in F} L(h, P)$ is the approximation error, while the $[dim(F)/m]$ term is the estimation error. A larger network with more trainable weights reduces the approximation error. But the dimensionality of the network scales with the number of weights, and hence the estimation error increases with the size of the network. Fig. 1 shows an oversized off-the shelf linear network (https://www.tensorflow.org/datasets/keras_example) for the MNIST dataset.

Fig. 2 shows[1] the training and test cross-entropy loss and accuracy for the network across epochs during gradient descent training. Increasing the number of training epochs improves the training loss and accuracy, but the test loss improves for the first few epochs and then degrades due to overfitting. On extremely large networks, the test loss may degrade after one epoch, resulting in poor utilization of the training samples, Xue et al. (2023).

To improve upon the above, we first analyze overfitting in the context of Equation 2. Gradient descent training (a) initializes the weights in the network to small random values and then (b) operating on the training samples, amplifies weights that improve the training loss most. Additional training epochs repeat step (b) on the training samples. Fig. 3 is a conceptual visualization of gradient descent. The

[1]Throughout this paper, error bars are twice the maximum standard error of the mean across each plot, indicative of variability across the training runs.

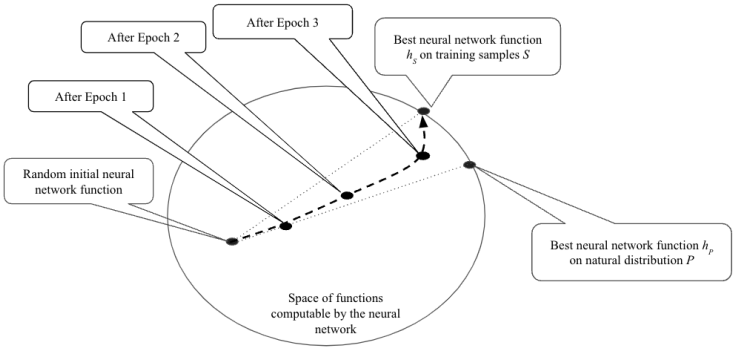


Figure 3: Conceptual visualization of gradient descent across epochs

ball is the space F of all functions computable by the neural network. There are two points of interest on the boundary of the ball. First, the function computed by the neural network that minimizes the loss on the natural distribution, i.e. the minimum-loss hypothesis $h_P = \text{argmin}_{h \in F} L(h, P)$. Second, the function computed by the neural network instance that minimizes the loss on the training samples, i.e. $h_S = \text{argmin}_{h \in F} L(h, S)$. In other words, h_P minimizes the test loss, while h_S minimizes the training loss. The weights of the network are randomly initialized to compute a function $f_0 \in F$. Thereafter, gradient descent iterates over the training samples and adjusts the weights to reduce the training loss so that the network computes functions $f_1, f_2, f_3 ... f_i$ after successive epochs. During the first epoch, the function computed by the network moves closer to h_P. In later epochs, the training samples are repeated, thereby pulling the network towards h_S. In brief, the training loss, which is represented by the distance between f_i and h_S, decreases across epochs. The test loss, which is represented by the distance between f_i and h_P, initially decreases and then increases, causing the overfitting phenomenon of Fig. 2.

We seek to improve gradient descent so that we can train the network for multiple epochs without overfitting, thereby extracting more of the information in the training samples. To that end, we first define a well-conditioned pruning algorithm as follows.

Definition 4. *A well-conditioned pruning algorithm takes as input a neural network computing a function f in a space of functions F and a pruning step size $\lambda \in [0, \hat{\lambda}] \subseteq [0, 1]$, and produces as output a reduced network $G \subset F$, such that (a) the dimension of the space is reduced by a fraction λ, i.e.,*

$$dim(G) \leq (1 - \lambda) dim(F) \tag{3}$$

and (b) satisfies a Lipschitz condition in that there exists $c \geq 0$ such that

$$\min_{h \in G} L(h, S) - \min_{h \in F} L(h, S) \leq c\lambda |L(f, S) - \min_{h \in F} L(h, S)| \tag{4}$$

Equation 4 bounds the error introduced by pruning via the Lipschitz contstant c over the interval $[0, \hat{\lambda}]$. The pruning algorithm may use any means to reduce the space of functions computed by the network, e.g., remove, freeze or quantize weights. With the foregoing in hand, we interleave training and pruning to improve performance and efficiency. Specifically, starting with $F = F_0$, we alternate training and pruning such that $F_0 \supset F_1 \supset F_2 ... \supset F_i$, where F_i is the space of functions computed by the network after pruning at the ith epoch, and f_i is the function computed by the neural network after training at the ith epoch. This is formalized in the Occam Pruning Algorithm below, and provably good per Lemma 1. The algorithm refers to τ_i as the "control" loss, which is the expected test loss at epoch i as estimated over a small fraction of the training samples held back during training. Also, let τ_∞ denote the algorithm's expected test loss at termination or in the limit.

Lemma 1. *If the initial pruning rate $\lambda_0 > 0$ is such that $\tau_3 < \tau_2 < \tau_1$, and the pruning algorithm is well-conditioned, (a) τ_i converges monotonically; (b) the expected size of the network in terms of its dimension decreases by a factor no less than $e^{-\lambda_0(2+(\tau_\infty-\tau_2)/(\tau_2-\tau_1)}$; and (c) if the Lipschitz constant of the pruning algorithm $c < 1/2$ over the interval $[0, \lambda_0/(\tau_1/\tau_2 - 1)]$, τ_∞ is less than the expected test loss of the underlying training algorithm.*

Proof. **Part (a)** We proceed by induction. Basis: $\tau_3 < \tau_2 < \tau_1$; Induction: assume $\tau_i < \tau_{i-1} < \tau_{i-2}$ and show that $\tau_{i+1} < \tau_i$. Now

$$\lambda_i = \frac{\tau_i - \tau_{i-1}}{\tau_{i-1} - \tau_{i-2}}\lambda_{i-1} = \frac{(\tau_i - \tau_{i-1})(\tau_{i-1} - \tau_{i-2})...(\tau_3 - \tau_2)}{(\tau_{i-1} - \tau_{i-2})...(\tau_3 - \tau_2)(\tau_2 - \tau_1)}\lambda_0 = \frac{\tau_i - \tau_{i-1}}{\tau_2 - \tau_1}\lambda_0 \tag{5}$$

Also, $\lambda_0 > 0$, per the inductive hypothesis $\tau_i - \tau_{i-1} < 0$, and per the basis $\tau_2 - \tau_1 < 0$. It follows that $\lambda_i > 0$. Next, per Equation 2, after the training step at each epoch, the expected test loss is asymptotically of the following form, where $m = |S|$ is the number of samples and C is a positive constant.

$$\tau_i = L(f_i, P) = \min_{h \in F_{i-1}} L(h, P) + C(dim(F_{i-1})/m)^{0.5} \tag{6}$$

After pruning, per Equation 4

$$\min_{h \in F_i} L(h, S) - \min_{h \in F_{i-1}} L(h, S) \leq c\lambda_i[(L(f_i, S) - \min_{h \in F_{i-1}} L(h, S))]$$

Taking expectations on both sides of the above, we get

$$\min_{h \in F_i} L(h, P) - \min_{h \in F_{i-1}} L(h, P) \leq c\lambda_i[(L(f_i, P) - \min_{h \in F_{i-1}} L(h, P))]$$

Substituting Equation 6 in the right-hand-side above, we get

$$\min_{h \in F_i} L(h, P) - \min_{h \in F_{i-1}} L(h, P) \leq c\lambda_i C(dim(F_{i-1})/m)^{0.5} \tag{7}$$

Incrementing epoch index i in Equation 6 we get

$$\tau_{i+1} = L(f_{i+1}, P) = \min_{h \in F_i} L(h, P) + C(dim(F_i)/m)^{0.5} \tag{8}$$

Substituting Equations 3 and 7 in the above, we get

$$\tau_{i+1} \leq \min_{h \in F_{i-1}} L(h, P) + c\lambda_i C(dim(F_{i-1})/m)^{0.5} + C(dim(F_{i-1})/m)^{0.5}(1 - \lambda_i)^{0.5}$$

Substituting the Taylor series inequality $(1 - \lambda_i)^{0.5} \leq (1 - \lambda_i/2)$ and rearranging, we get

$$\tau_{i+1} \leq [\min_{h \in F_{i-1}} L(h, P) + C(dim(F_{i-1})/m)^{0.5}] + \lambda_i C(c - 1/2)(dim(F_{i-1})/m)^{0.5}$$

Substituting Equation 6 in the above and rearranging, we get

$$\tau_{i+1} - \tau_i \leq \lambda_i C(c - 1/2)(dim(F_{i-1})/m)^{0.5} \tag{9}$$

Likewise, we obtain

$$\tau_i - \tau_{i-1} \leq \lambda_{i-1} C(c - 1/2)(dim(F_{i-2})/m)^{0.5} \tag{10}$$

Substituting $\lambda_i = \lambda_{i-1}(\tau_i - \tau_{i-1})/(\tau_{i-1} - \tau_{i-2})$ per the algorithm in Equation 9 we get

$$\tau_{i+1} \leq \tau_i + \lambda_{i-1}\frac{\tau_i - \tau_{i-1}}{\tau_{i-1} - \tau_{i-2}} C(c - 1/2)(dim(F_{i-1})/m)^{0.5}$$

Substituting Equation 10 in the above,

$$\tau_{i+1} \leq \tau_i + \frac{\lambda_{i-1}^2 C^2 (c - 1/2)^2}{\tau_{i-1} - \tau_{i-2}} dim(F_{i-1})/m)^{0.5} dim(F_{i-2})/m)^{0.5} \tag{11}$$

Earlier we showed that $\lambda_{i-1} > 0$. Now $dim(F_{i-2}) > 0$ else $\tau_{i-1} = \tau_{i-2}$ and the algorithm would have terminated. Likewise $dim(F_{i-1}) > 0$; and by the inductive assumption $\tau_{i-1} < \tau_{i-2}$. Hence the second term on the right-hand-side above is strictly negative implying $\tau_{i+1} < \tau_i$. And per the inductive basis, $\tau_3 < \tau_2 < \tau_1$, it follows that $\tau_{i+1} < \tau_i$.

Part (b) The final dimension of the network as a multiple of the initial dimension is as follows, using the inequality $1 - x \leq e^{-x}$.

$$\prod_{k=1}^{\infty}(1-\lambda_k) = (1-\lambda_0)^2 \prod_{k=3}^{\infty}(1-\lambda_k) \leq e^{-2\lambda_0 - \sum_{k=3}^{\infty}\lambda_k}$$

Using Equation 5, we can write

$$2\lambda_0 + \sum_{k=3}^{\infty}\lambda_k = 2\lambda_0 + \sum_{k=3}^{\infty}\frac{\tau_k - \tau_{k-1}}{\tau_2 - \tau_1}\lambda_0 = 2\lambda_0 + \frac{\tau_\infty - \tau_2}{\tau_2 - \tau_1}\lambda_0 = \lambda_0\left(2 + \frac{\tau_\infty - \tau_2}{\tau_2 - \tau_1}\right).$$

Hence the final dimension of the network is at most $e^{-\lambda_0(2+(\tau_\infty - \tau_2)/(\tau_2 - \tau_1)}$ the initial dimension.

Part (c) Returning to Equation 6 and recursively invoking Equations 3 and 4, we get

$$\begin{aligned}
\tau_i = L(f_i, P) &= \min_{h \in F_{i-1}} L(h, P) + C(dim(F_{i-1})/m)^{0.5} \\
&\leq \min_{h \in F_{i-2}} L(h, P) + \lambda_{i-1}C(c - 1/2)(dim(F_{i-2})/m)^{0.5} + C(dim(F_{i-2})/m)^{0.5} \\
&... \\
&\leq \min_{h \in F_0} L(h, P) + \sum_{k=1}^{i-1}\lambda_k C(c - 1/2)(dim(F_{k-1})/m)^{0.5} + C(dim(F_0)/m)^{0.5} \qquad (12)
\end{aligned}$$

Since $\tau_i < \tau_{i-1}...\tau_1$ for all i, Equation 5 implies that $\lambda_i < \lambda_0\tau_2/(\tau_1 - \tau_2)$. Therefore if $c < 1/2$ over the interval $[0, \lambda_0/(\tau_1/\tau_2 - 1)]$, we have

$$C(c - 1/2)\sum_{k=2}^{i-1}\lambda_k(dim(F_{k-1})/m)^{0.5} \leq -C|1/2 - c|\lambda_0(dim(F_0)/m)^{0.5}$$

Substituting $F_0 = F$ and combining the above with Equation 12 we get

$$\tau_\infty = \lim_{i \to \infty}\tau_i \leq [\min_{h \in F} L(h, P) + C(dim(F)/m)^{0.5}] - C|1/2 - c|\lambda_0(dim(F)/m)^{0.5} \qquad (13)$$

Noting that the first term on the right of the above inequality is the expected test loss of the training algorithm without pruning, and that the second term is strictly negative, yields the result. □

The additional hyperparameter for the initial pruning rate λ_0 is a limitation of our algorithm. If the underlying training algorithm is not optimal, an arbitrarily small λ_0 can be chosen to satisfy the condition of the lemma. However, larger values of λ_0 improve performance per Equation 13. Hence, as with all adaptive training, e.g., Kingma & Ba (2014), the choice of the initial pruning rate plays an important role.

The algorithm refers to τ_i as the control loss, which is the test loss over a small fraction of the training samples held back during training. In our experiments of the next section, using the training loss for control performed just as well as using the loss on a holdback subset of training samples. In practice, we cannot assert that popular pruning algorithms are well-conditioned per our definition above. Nevertheless, we use Occam Pruning to combine gradient descent training with the popular magnitude pruning into "Occam Gradient Descent," where at each pruning step, the smallest multiplicative weights by magnitude are clamped to zero, leaving the bias weights untouched. Specifically, for pruning step size λ, multiplicative weights smaller in absolute value than the λ-quantile of each layer are clamped to zero, where the λ-quantile of a distribution is the value q such that λ is the mass of the distribution below q. For example, for $\lambda = 0.5$, q is the median. Pruning can be based on other measures of relative importance amongst the weights; and other measures of the dimension of a network, e.g. the norm of the weights as in regularization, Bartlett (1996). In practice, we found that stability is improved by constraining λ to a positive interval, e.g. $[\lambda_0/10, \lambda_0]$. We also found that on large training sets, the pruning step in the algorithm can be performed at fractional epochs. Furthermore, several variants of the algorithm are possible such as restoring a small random fraction of the weights that were set to zero previously. Fig. 4 shows the loss and accuracy with Occam Gradient Descent on the network of Fig. 1 for MNIST. Comparing Fig. 4 with Fig. 2, it is clear that the algorithm resists overfitting to substantially improve test loss and

Occam Pruning Algorithm
Input: *neural network, samples S*
Parameters: *training algorithm, pruning algorithm, initial pruning rate* $\lambda_0 \in [0, 1]$

Initialize network at random
Start: *let* $\lambda_2 \leftarrow \lambda_1 \leftarrow \lambda_0$
For *epoch* $i = 1, 2, \ldots$
Train the network with samples S to get control loss τ_i
If τ_i *equals* τ_{i-1}*: output current network & stop*
Else if $i > 2$*:*
$\lambda_i \leftarrow \lambda_{i-1}\,(\tau_i - \tau_{i-1})/(\tau_{i-1} - \tau_{i-2})$
Prune the network by step λ_i

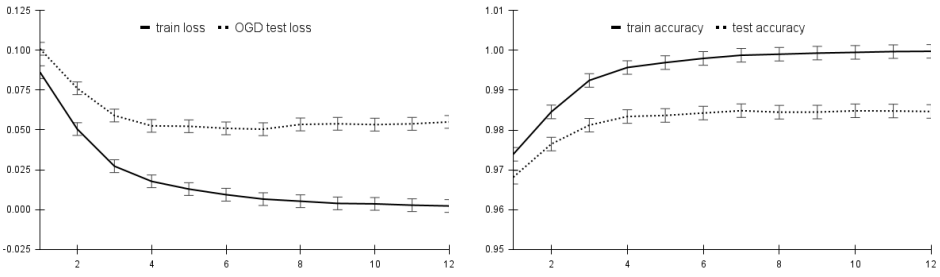


Figure 4: Occam Gradient Descent loss & accuracy; MNIST; average of ten runs; $\lambda = 0.4$

accuracy. Furthermore, since the model size decreases with epochs, the total computation cost of training is reduced as noted in the next section. If training is continued for many epochs after the test loss levels off, eventually the model gets so small that the conditions of Lemma 1 no longer hold, At that point, the test loss explodes.

In brief, Occam Gradient Descent interleaves adaptive reduction of model size to minimize the generalization error, with gradient descent on model weights to minimize training loss. In contrast, traditional gradient descent greedily minimizes training loss without regard to the generalization error.

3 Experimental Results

Unless specified otherwise, all experiments in this paper used magnitude pruning of the multiplicative weights and gradient descent with the Adam optimizer, running on an M1-Mac laptop. Our limited computational resources restricted us from testing the largest models.

Table 1 compares the performance of the Occam Gradient Descent algorithm on the MNIST network of Fig.1 over twelve epochs, across ten runs. For each algorithm, the table shows the statistics averaged at the epochs with the minimum test loss for each run. We remind the reader that the average at the minimum test loss is distinct from the minimum of the average test loss of Fig. 2 and Fig. 4. The first row shows the statistics for Gradient Descent. The second row shows the statistics for the Occam Gradient Descent algorithm at an initial pruning rate $\lambda_0 = 0.4$ and 10% holdback for the control loss. For this algorithm, at the minimum test loss, the average size of the network is $\approx 23\%$ of the original in terms of the number of non-zero weights. The projected compute cost is the average cumulative cost of the epochs across the shrinking network. The third row shows the statistics for Occam Gradient Descent without any holdback, but using the training loss for control. The last row shows the performance of conventional post-train pruning: 6 epochs of gradient descent followed by pruning to target size of $\approx 21\%$, and then retraining for 6 epochs. The 6 epochs are

Table 1: Performance on MNIST network of Fig.1; 12 epochs; 10 run average at best test loss

Algorithm	training loss	Train Acc.	Test Loss	Test Acc.	Size	Compute
Gradient Descent	0.0187	99.4%	0.066	98.1%	100%	5.9
Occam Gradient Descent (10% holdback loss, $\lambda_0 = 0.4$)	0.0066	99.9%	0.05	98.5%	23%	3.1
Occam Gradient Descent (training loss, $\lambda_0 = 0.4$)	0.008	99.9%	0.049	98.5%	21%	2.8
Conventional post-train pruning	0.0024	99.9%	0.056	98.5%	21%	7.1

Layer (type)	Output Shape	Param #
conv2d (Conv2D)	(None, 30, 30, 64)	1,792
max_pooling2d (MaxPooling2D)	(None, 15, 15, 64)	0
conv2d_1 (Conv2D)	(None, 13, 13, 128)	73,856
max_pooling2d_1 (MaxPooling2D)	(None, 6, 6, 128)	0
conv2d_2 (Conv2D)	(None, 4, 4, 128)	147,584
flatten (Flatten)	(None, 2048)	0
dense (Dense)	(None, 128)	262,272
dense_1 (Dense)	(None, 10)	1,290

Total params: 486,794 (1.86 MB)
Trainable params: 486,794 (1.86 MB)
Non-trainable params: 0 (0.00 B)

Figure 5: Oversized off-the shelf convolutional network for CIFAR10

the rounded average number of epochs at which traditional gradient descent minimized test loss, per the "Compute" column of the first row in the table. It is evident that Occam Gradient Descent outperforms the other approaches at a lower computational cost, lower test loss, and smaller model size.

Fig. 5 shows an oversized off-the shelf network for the CIFAR10 dataset combining both linear units and convolutional units. Fig. 6 shows the corresponding results comparing the performance of gradient descent and Occam Gradient Descent with training loss control. As for MNIST in Table 1, 10% holdback loss control showed similar performance. Fig. 5 also shows the fractional size of the model by epoch. It is clear that Occam Gradient Descent resists overfitting and outperforms gradient descent, even while reducing the network to a small fraction of its original size.

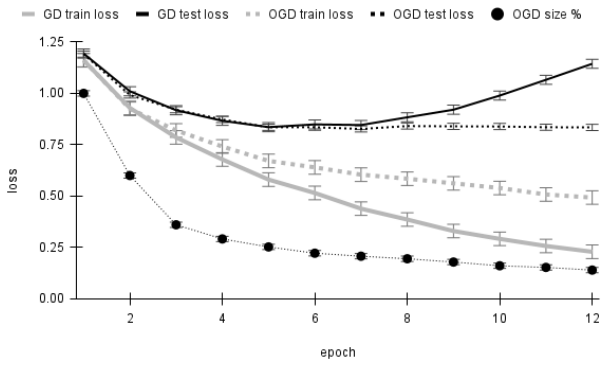


Figure 6: Gradient Descent (GD) & Occam Gradient Descent (OGD); CIFAR10; training loss control; average of 10 runs; $\lambda_0 = 0.4$

Layer (type)	Output Shape	Param #
flatten (Flatten)	(None, 21)	0
dense (Dense)	(None, 512)	11,264
dropout (Dropout)	(None, 512)	0
dense_1 (Dense)	(None, 2)	1,026

Total params: 12,290 (48.01 KB)
Trainable params: 12,290 (48.01 KB)
Non-trainable params: 0 (0.00 B)

Figure 7: Dense linear network of 512 units for classification of tabular data with 21 input features

Table 2: Tabular Classification: GD & OGD Neural Networks; Random Forests (10 run average)

			Random Forest		GD Network		OGD Network	
Dataset	Samples	Features	Size (nodes)	Test Loss	Size (wts)	Test Loss	Size(wts)	Test Loss
Census Income	48,842	14	544,120	0.429	8,706	0.311	1,745	0.308
Breast Cancer	569	30	5,872	0.236	16,898	0.103	3,082	0.119
Heart Disease	303	13	11,754	0.367	8,194	0.368	1,893	0.374
Credit Card Default	30,000	23	546,372	0.521	13,314	0.441	2,448	0.434
Room Occupancy	10,129	18	7,630	0.003	10,754	0.005	2,707	0.003
CDC Diabetes	253,680	21	2,909,684	0.540	12,290	0.315	2,239	0.313

We now consider the classification of tabular data where an unknown feature must be predicted from a set of known features, a frequent application of machine learning. For example, given a set of patient vitals, diagnose the disease. Such problems are commonly addressed via Boosted Trees and Random Forests, see for example, Sutton (2005), Biau & Scornet (2016). While Boosted Trees and Random Forests naturally resist overfitting, they typically create large models that scale up with the size of the data set. In contrast, deep learning models are relatively compact but subject to overfitting when trained via gradient descent. Since Occam Gradient Descent addresses the overfitting limitation of deep learning networks, we test its applicability to classifying tabular data. Specifically, we compare the performance of Random Forests against a simple neural network with 512 dense linear units trained on tabular data sets. Fig. 7 shows the network for a data set with 21 input features.

Table 2 compares the performance of Random Forests and neural networks trained on a range of binary tabular classification data sets from the UC Irvine repository (https://archive.ics.uci.edu/datasets). Each data set was randomly split into a training set comprising 75% of the samples and a test set of the remaining samples, the split being fixed across runs. For Random Forests, we used the default settings in TensorFlow. On each data set, neural networks of the form of Fig. 7 were trained for 12 epochs with Gradient Descent (GD Neural Network), and Occam Gradient Descent (OGD Neural Network) with pruning rate $\lambda_0 = 0.4$ using training loss for control. Table 2 reports averages at the final epoch across ten runs. Fig. 8 is a visual summary of Table 2. In brief, compared to Random Forests on average across the data sets: neural networks trained with Gradient Descent are $\approx 16\%$ smaller at $\approx 11\%$ better cross-entropy loss; while neural networks trained with Occam Gradient Descent are $\approx 80\%$ smaller at $\approx 20\%$ better cross-entropy loss.

Next, we apply Occam Gradient Descent to natural language transformer models. Specifically, the open source nanoGPT model (https://github.com/karpathy/nanoGPT/blob/master/README.md) of Table 3 with dropout = 0.2 trained on Shakespeare's works, with a training set of 1M tokens, and

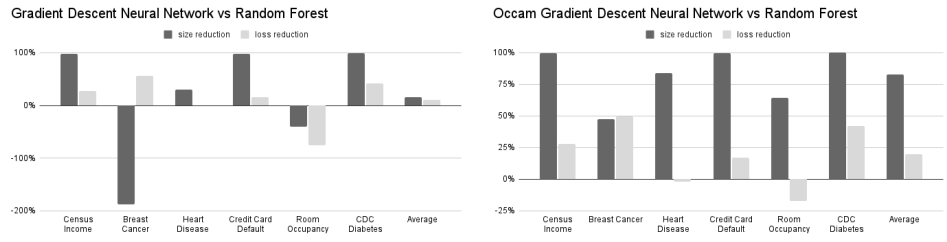


Figure 8: Size and loss reduction on tabular data sets

Table 3: nanoGPT LLM with $\approx 11M$ parameters on Shakespeare's works

Layers	Heads	Context Length	Embed Dimension
6	6	128	384

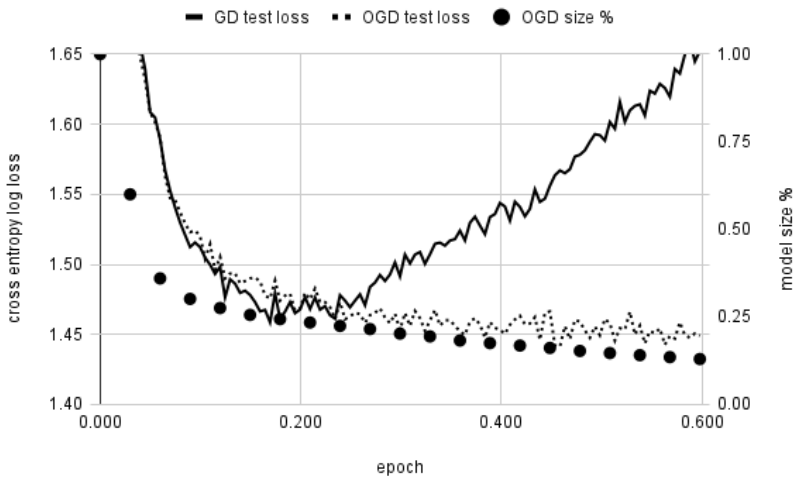


Figure 9: Gradient Descent (GD) & Occam Gradient Descent (OGD) on LLM

a test set of $\approx 100K$ tokens. Fig. 9 shows the test loss for Gradient Descent and the test loss and model size for Occam Gradient Descent against training epochs. Using training loss control, the initial pruning rate $\lambda_0 = 0.4$, and the contraction step is applied at intervals of 0.03 epochs. Under gradient descent training, the model overfits and the test loss is a minimum of 1.4587 at 0.18 epochs, rising with further training. Under Occam Gradient Descent, both the test loss and the model size continue to improve with training. The test loss under Occam Gradient Descent surpasses that of Gradient Descent at 0.264 epochs, at which point the model size is 20% of its original size in terms of the number of non-zero weights. Reflective of the declining model size, the compute required by Occam Gradient descent for 0.264 epochs is 60% of the compute effort required by Gradient Descent to achieve its minimum loss at 0.18 epochs. We note that this experiment uses the AdamW optimizer and that OGD performs on top of the built-in regularization.

4 SUMMARY

Deep learning neural network models must be large enough to adapt to their problem domain, while small enough to avoid overfitting training data during gradient descent. To balance these competing demands, overprovisioned deep learning models such as transformers are trained for a single epoch on large data sets, and hence inefficient with both computing resources and training data. In response to these inefficiencies, we derive a provably good algorithm that combines training and pruning to simultaneously optimize efficiency and accuracy, identifying conditions that resist overfitting and reduce model size while outperforming the underlying training algorithm. We then use the algorithm to combine gradient descent with magnitude pruning into "Occam Gradient Descent." With respect to loss, compute and model size our algorithm outperforms traditional gradient descent on a broad range of data sets.

REFERENCES

T. Asami, R. Masumura, Y. Yamaguchi, H. Masataki, and Y. Aono. Domain adaptation of dnn acoustic models using knowledge distillation. In *2017 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*, pp. 5185–5189. IEEE, 3 2017.

G. Biau and E. Scornet. A random forest guided tour. *Test*, 25:197–227, 2016.

D. Blalock, J. J. Gonzalez Ortiz, J. Frankle, and J. Guttag. What is the state of neural network pruning? *Proceedings of machine learning and systems*, 2:129–146, 2020.

A. Blumer, A. Ehrenfeucht, D. Haussler, and M. K. Warmuth. Occam's razor. *Information processing letters*, 24(6):377–380, 1987.

R. Board and L. Pitt. On the necessity of occam algorithms. In *Proceedings of the twenty-second annual ACM symposium on Theory of computing*, pp. 54–63. ACM, 4 1990.

T. Brown, B. Mann, N. Ryder, M. Subbiah, J. D. Kaplan, P. Dhariwal, others, and D. Amodei. Language models are few-shot learners. *Advances in neural information processing systems*, 33: 1877–1901, 2020.

X. Chang, Y. Li, S. Oymak, and C. Thrampoulidis. Provable benefits of overparameterization in model compression: From double descent to pruning neural networks. In *Proceedings of the AAAI Conference on Artificial Intelligence (Vol*, pp. 6974–6983, No. 8, 5 2021. 35.

G. Chen, W. Choi, . X. Yu, T. Han, and M. Chandraker. Learning efficient object detection models with knowledge distillation. *Advances in neural information processing systems*, 30, 2017.

J. Frankle and M. Carbin. The lottery ticket hypothesis: Finding sparse, trainable neural networks. *arXiv:1803.03635*, 2018.

E. Frantar and D. Alistarh. Sparsegpt: Massive language models can be accurately pruned in one-shot. In *International Conference on Machine Learning*, pp. 10323–10337. PMLR, 7 2023.

R. Ghosh and M. Motani. Network-to-network regularization: enforcing occam's razor to improve generalization. *Advances in neural information processing systems*, 34:6341–6352, 2021.

S. Han, J. Pool, J. Tran, and W. Dally. Learning both weights and connections for efficient neural network. *Advances in neural information processing systems*, 28, 2015.

Babak Hassibi, David G Stork, and Gregory J Wolff. Optimal brain surgeon and general network pruning. In *IEEE international conference on neural networks*, pp. 293–299. IEEE, 1993.

D. Haussler. Decision theoretic generalizations of the pac model for neural net and other learning applications. *Information and Computation*, 100(1):78–150, 1992.

G. Hinton, O. Vinyals, and J. Dean. Distilling the knowledge in a neural network. *arXiv:1503.02531*, 2015.

T. Hoefler, D. Alistarh, T. Ben-Nun, N. Dryden, and A. Peste. Sparsity in deep learning: Pruning and growth for efficient inference and training in neural networks. *Journal of Machine Learning Research*, 22(241):1–124, 2021.

J. Hoffmann, S. Borgeaud, A. Mensch, E. Buchatskaya, T. Cai, E. Rutherford, others, and L. Sifre. Training compute-optimal large language models. *arXiv:2203.15556*, 2022.

Jared Kaplan, Sam McCandlish, Tom Henighan, Tom B. Brown, Benjamin Chess, Rewon Child, Scott Gray, Alec Radford, Jeffrey Wu, and Dario Amodei. Scaling laws for neural language models. *arXiv:2001.08361*, 2020.

B. N. Kausik. Scaling efficient llms. *arXiv:2402.14746*, 2024.

D. P. Kingma and J. Ba. Adam: A method for stochastic optimization. *arXiv:1412.6980*, 2014.

J. Kukačka, V. Golkov, and D. Cremers. Regularization for deep learning: A taxonomy. *arXiv:1710.10686*, 2017.

Yann LeCun, John Denker, and Sara Solla. Optimal brain damage. In *Advances in neural information processing systems 2 Hassibi, B*, pp. 293–299, D. G., & Wolff, G. J. (1993, March). Optimal brain surgeon and general network pruning. In IEEE international conference on neural networks . IEEE, 1989. Stork.

Z. Liu, M. Sun, T. Zhou, G. Huang, and T. Darrell. Rethinking the value of network pruning. *arXiv:1810.05270*, 2018.

D. C. Mocanu, E. Mocanu, P. Stone, P. H. Nguyen, M. Gibescu, and A. Liotta. Scalable training of artificial neural networks with adaptive sparse connectivity inspired by network science. *Nature communications*, 9(1):2383, 2018.

B. K. Natarajan. On learning sets and functions. *Machine Learning*, 4:67–97, 1989.

B. K. Natarajan. Occam's razor for functions. In *Proceedings of the sixth annual conference on Computational learning theory*, pp. 370–376. ACM, 8 1993.

J. W. Rae, S. Borgeaud, T. Cai, K. Millican, J. Hoffmann, F. Song, others, and G. Irving. Scaling language models: Methods, analysis & insights from training gopher. *arXiv:2112.11446*, 2021.

S. Shalev-Shwartz and S. Ben-David. *Understanding machine learning*. university press, Cambridge, 2014.

S. Smith, M. Patwary, B. Norick, P. LeGresley, S. Rajbhandari, J. Casper, others, and B. Catanzaro. Using deepspeed and megatron to train megatron-turing nlg 530b, a large-scale generative language model. *arXiv:2201.11990*, 2022.

M. Sun, Z. Liu, A. Bair, and J. Z. Kolter. A simple and effective pruning approach for large language models. *arXiv:2306.11695*, 2023.

Clifton D Sutton. Classification and regression trees, bagging, and boosting. *Handbook of statistics*, 24:303–329, 2005.

R. Thoppilan, D. De Freitas, J. Hall, N. Shazeer, A. Kulshreshtha, H. T. Cheng, others, and Q. Le. Lamda: Language models for dialog applications. *arXiv:2201.08239*, 2022.

R. Tibshirani. Regression shrinkage and selection via the lasso. *Journal of the Royal Statistical Society Series B: Statistical Methodology*, 58(1):267–288, 1996.

L. G. Valiant. A theory of the learnable. *Communications of the ACM*, 27(11):1134–1142, 1984.

Colin White, Mahmoud Safari, Rhea Sukthanker, Binxin Ru, Thomas Elsken, Arber Zela, Debadeepta Dey, and Frank Hutter. Neural architecture search: Insights from 1000 papers. *arXiv preprint arXiv:2301.08727*, 2023.

F. Xue, Y. Fu, W. Zhou, Z. Zheng, and Y. You. To repeat or not to repeat: Insights from scaling llm under token-crisis. *arXiv:2305.13230*, 2023. arXiv preprint.

L. Yuan, F. E. Tay, G. Li, T. Wang, and J. Feng. Revisiting knowledge distillation via label smoothing regularization. In *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*, pp. 3903–3911, 2020.

Z. Zhang, R. Tao, and J. Zhang. Neural network pruning by gradient descent. *arXiv:2311.12526*, 2023.